

RaccoonBot OpenVLA Pipeline 확장 보고서

Cube 객체 추가, Push Task 학습 및 실제 RaccoonBot 실행

2021741037 정민승

1. 서론

본 과제의 목표는 기존 RaccoonBot OpenVLA pipeline을 확장하여, 단순한 colored cylinder grasp task를 더 다양한 object와 task로 확장하는 것이다. 기존 예제는 MuJoCo 환경에서 색상이 다른 cylinder를 대상으로 "grasp the red cylinder"와 같은 language instruction을 입력받고, OpenVLA 모델이 이를 바탕으로 grasp action을 예측하는 구조였다.

본 프로젝트에서는 기존 pipeline에 cube object를 추가하고, grasp task뿐만 아니라 push task까지 수행할 수 있도록 MuJoCo scene, dataset generation code, local inference client를 수정하였다. 최종적으로 green/yellow cylinder와 red/blue cube를 포함하는 4-object scene을 구성하였고, grasp task 4종류와 cube push task 2종류로 구성된 총 300개의 successful demonstration dataset을 생성하였다.

이후 생성한 dataset을 RLDS/TFDS format으로 변환하고, OpenVLA base model을 LoRA 방식으로 fine-tuning하였다. Fine-tuning이 완료된 모델은 local MuJoCo client에서 grasp/push task를 수행하도록 테스트하였으며, 추가로 실제 RaccoonBot hardware와 연결하여 yellow cylinder grasp 동작까지 실행하였다.

2. 기존 Pipeline 분석

기존 RaccoonBot OpenVLA pipeline은 다음과 같은 순서로 구성된다.

1. MuJoCo simulation에서 RaccoonBot demonstration 생성
2. Raw episode를 RLDS intermediate format으로 변환
3. TFDS dataset build
4. OpenVLA LoRA fine-tuning
5. OpenVLA inference server 실행
6. Local MuJoCo client에서 prediction action 실행

기존 task는 color-conditioned cylinder grasping에 집중되어 있었다. 즉, scene에는 여러

색상의 cylinder가 존재하고, language instruction은 "grasp the {color} cylinder" 형태로 구성되었다. 이 구조는 VLA pipeline의 기본 흐름을 확인하기에는 적절하지만, object type이 cylinder로 제한되고 task도 grasp 하나만 존재한다는 한계가 있었다.

따라서 본 과제에서는 object type과 task type을 확장하여, 보다 다양한 language instruction과 action pattern을 학습하도록 구성하였다.

3. Dataset Extension

3.1 Object Extension

기존 scene은 colored cylinder 중심으로 구성되어 있었다. 본 프로젝트에서는 cube object를 추가하여 최종 active object를 다음 4개로 구성하였다.

Object	Shape	Color
--------	-------	-------

green cylinder	cylinder	green
----------------	----------	-------

yellow cylinder	cylinder	yellow
-----------------	----------	--------

red cube	cube	red
----------	------	-----

blue cube	cube	blue
-----------	------	------

이를 위해 MuJoCo XML 파일을 수정하여 cube body를 추가하고, dataset generation code에서 cylinder와 cube를 모두 target object로 사용할 수 있도록 object mapping을 확장하였다.

3.2 Task Extension

기존 task는 grasp only였지만, 본 프로젝트에서는 push task를 추가하였다. 최종 dataset은 다음 6개 task-target 조합으로 구성하였다.

Task	Target	Episodes
------	--------	----------

grasp	green cylinder	50
-------	----------------	----

grasp	yellow cylinder	50
-------	-----------------	----

grasp	red cube	50
-------	----------	----

Task Target	Episodes
grasp blue cube	50
push red cube	50
push blue cube	50
Total	300

Cylinder push도 추가로 실험하였으나, cylinder는 접촉 시 굴러가거나 옆으로 들어지는 현상이 자주 발생하였다. 특히 push distance를 크게 설정했을 때 안정적인 successful episode를 확보하기 어려웠다. 따라서 최종 dataset에서는 cylinder는 grasp task에 사용하고, push task는 red cube와 blue cube로 제한하였다.

4. 수정한 파일 및 역할

본 프로젝트에서 주요하게 수정하거나 추가한 파일은 다음과 같다.

File	Description
Mujoco/Raccoon_colored_cylinder_cube.xml	cylinder와 cube가 함께 포함되도록 수정한 MuJoCo scene
Mujoco/raccoon_grasp_push_4objects_alltargets_dataset_fixed.py	grasp/push demonstration dataset 생성 코드
local_client/openvla_grasp_push_4objects_custom_client_focus_pushboost.py	grasp/push inference 실행을 위한 local MuJoCo client

File	Description
local_client/opencvla_multicolor_client_real_robot.py	실제 RaccoonBot hardware 실행을 위한 client
logs/finetune_grasp_push_300ep_10000step_b2_ga8.log	OpenVLA LoRA fine-tuning log
Mujoco/gif/episode_000007.gif	grasp demonstration visualization
Mujoco/gif/episode_000238.gif	push demonstration visualization
Mujoco/gif/grasp.gif	fine-tuned model local grasp inference 결과
Mujoco/gif/push.gif	fine-tuned model local push inference 결과
Mujoco/gif/real.gif	실제 RaccoonBot yellow cylinder grasp 결과

raccoon_env.py는 local client 실행에 필요한 기본 environment 파일로 포함하였다. 본

프로젝트의 핵심 수정 내용은 dataset generation script, MuJoCo XML, grasp/push inference client, 그리고 실제 RaccoonBot 실행을 위한 real robot client 활용에 집중되어 있다.

5. Raw Demonstration Dataset 생성

Raw demonstration dataset은 MuJoCo simulation에서 rule-based trajectory를 실행하여 생성하였다. 각 episode는 image frame, robot state, end-effector pose, object pose, action sequence, success flag를 포함한다.

Raw demonstration dataset 생성 명령어는 다음과 같다.

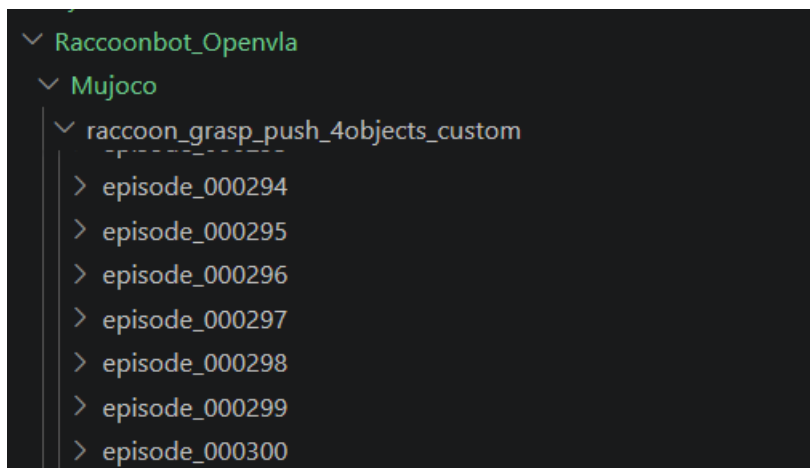
```
cd /data/Raccoonbot_Openvla/Mujoco
```

```
python raccoon_grasp_push_4objects_alltargets_dataset_fixed.py
```

생성된 raw dataset은 다음 경로에 저장하였다.

```
/data/Raccoonbot_Openvla/Mujoco/raccoon_grasp_push_4objects_custom
```

최종적으로 episode_000001부터 episode_000300까지 총 300개의 successful episode가 생성되었다.



6. RLDS / TFDS Dataset 변환

OpenVLA fine-tuning script는 raw MuJoCo episode를 직접 사용하지 않고, RLDS/TFDS 형식의 dataset을 사용한다. 따라서 생성된 raw episode를 RLDS intermediate format으로 변환한 뒤, TFDS dataset으로 build하였다.

6.1 RLDS Intermediate 변환

Raw MuJoCo episode를 RLDS intermediate format으로 변환하기 위해 다음 명령어를 사용하였다.

```
cd /data/Raccoonbot_Openvla/Mujoco/raccoon_dataset
```

```
rm -rf /data/Raccoonbot_Openvla/Mujoco/raccoon_dataset/openvla_rlds_intermediate
```

```
python convert_raw_to_openvla_rlds_intermediate.py ₩
```

```
--raw_root /data/Raccoonbot_Openvla/Mujoco/raccoon_grasp_push_4objects_custom ₩
```

```
--out_root
```

```
/data/Raccoonbot_Openvla/Mujoco/raccoon_dataset/openvla_rlds_intermediate ₩
```

```
--val_ratio 0.1
```

이 과정에서 raw episode는 train/validation split으로 나뉘고, OpenVLA dataset builder에서 사용할 수 있는 step-level record로 변환된다.

6.2 TFDS Build

RLDS intermediate data를 기반으로 TFDS dataset을 build하였다.

```
cd /data/Raccoonbot_Openvla/Mujoco/rlds_dataset_builder/raccoon_pick_place
```

```
tfds build --overwrite
```

생성된 TFDS dataset은 OpenVLA fine-tuning script가 읽을 수 있도록 project directory로 이동하였다.

```
rm -rf /data/Raccoonbot_Openvla/tensorflow_datasets
```

```
mv /root/tensorflow_datasets /data/Raccoonbot_Openvla/
```

최종 dataset 경로는 다음과 같다.

/data/Raccoonbot_Openvla/tensorflow_datasets

7. OpenVLA LoRA Fine-Tuning

본 프로젝트에서는 openvla/openvla-7b base model을 사용하고, 전체 모델을 full fine-tuning하는 대신 LoRA adapter를 학습하였다. LoRA fine-tuning은 대규모 모델의 대부분 parameter를 고정한 상태에서 adapter parameter만 학습하므로, 제한된 GPU 환경에서도 효율적으로 task adaptation을 수행할 수 있다.

Fine-tuning configuration은 다음과 같다.

Item	Value
Base model	openvla/openvla-7b
Dataset	raccoon_pick_place
LoRA rank	16
Batch size	2
Gradient accumulation steps 8	
Effective batch size	16
Learning rate	5e-4
Max steps	10,000
Save steps	1,000

Fine-tuning 명령어는 다음과 같다.

```
cd /data/Raccoonbot_Openvla/openvla
```

```
export PYTHONPATH=/data/Raccoonbot_Openvla/openvla:$PYTHONPATH
```

```
PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128 ₩
```

```
WANDB_MODE=disabled CUDA_VISIBLE_DEVICES=0 ₩
```

```

torchrun --standalone --nnodes 1 --nproc-per-node 1 vla-scripts/finetune.py ₩

--vla_path openvla/openvla-7b ₩

--data_root_dir /data/Raccoonbot_Openvla/tensorflow_datasets ₩

--dataset_name raccoon_pick_place ₩

--run_root_dir /data/Raccoonbot_Openvla/openvla/openvla-runs ₩

--adapter_tmp_dir /data/Raccoonbot_Openvla/openvla/openvla-adapter-tmp ₩

--lora_rank 16 ₩

--batch_size 2 ₩

--grad_accumulation_steps 8 ₩

--learning_rate 5e-4 ₩

--max_steps 10000 ₩

--save_steps 1000 ₩

--run_id_note raccoon-grasp-push-300ep-10000step-b2-ga8-r16 ₩

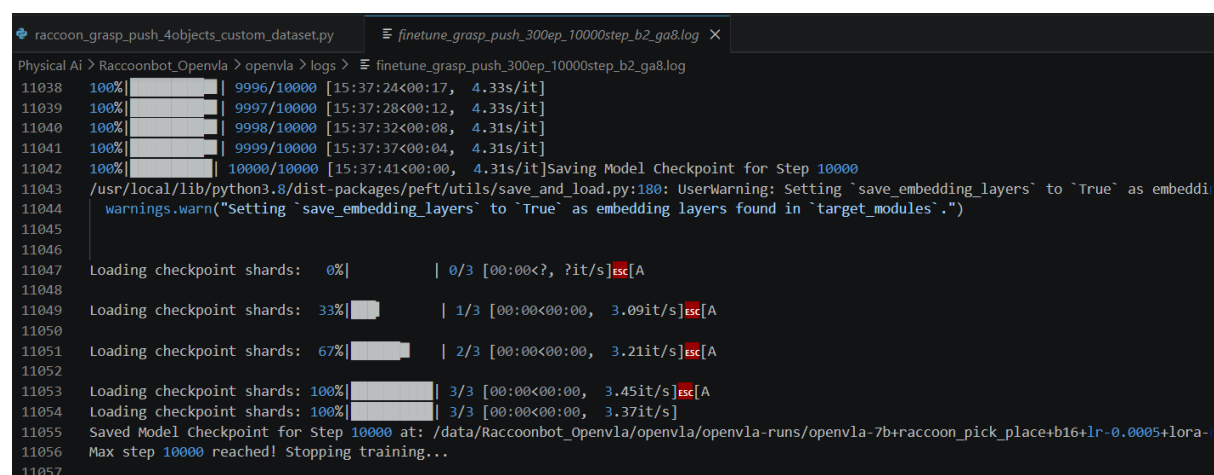
2>&1 | tee
/data/Raccoonbot_Openvla/openvla/logs/finetune_grasp_push_300ep_10000step_b2_ga8.log
og

```

학습은 10,000 step까지 정상적으로 완료되었으며, 최종 checkpoint가 저장되었다.

Max step 10000 reached.

Saved Model Checkpoint for Step 10000.



```

raccoon_grasp_push_4objects_custom_dataset.py  finetune_grasp_push_300ep_10000step_b2_ga8.log X
Physical Ai > Raccoonbot_Openvla > openvla > logs > finetune_grasp_push_300ep_10000step_b2_ga8.log
11038 100%|██████████| 9996/10000 [15:37:24<00:17, 4.33s/it]
11039 100%|██████████| 9997/10000 [15:37:28<00:12, 4.33s/it]
11040 100%|██████████| 9998/10000 [15:37:32<00:08, 4.31s/it]
11041 100%|██████████| 9999/10000 [15:37:37<00:04, 4.31s/it]
11042 100%|██████████| 10000/10000 [15:37:41<00:00, 4.31s/it]Saving Model Checkpoint for Step 10000
11043 /usr/local/lib/python3.8/dist-packages/peft/utils/save_and_load.py:180: UserWarning: Setting `save_embedding_layers` to `True` as embeddi
warnings.warn("Setting `save_embedding_layers` to `True` as embedding layers found in `target_modules`.")
11044
11045
11046
11047 Loading checkpoint shards: 0%|          | 0/3 [00:00<?, ?it/s]esc[A
11048
11049 Loading checkpoint shards: 33%|███      | 1/3 [00:00<00:00, 3.09it/s]esc[A
11050
11051 Loading checkpoint shards: 67%|██████   | 2/3 [00:00<00:00, 3.21it/s]esc[A
11052
11053 Loading checkpoint shards: 100%|████████| 3/3 [00:00<00:00, 3.45it/s]esc[A
11054 Loading checkpoint shards: 100%|████████| 3/3 [00:00<00:00, 3.37it/s]
11055 Saved Model Checkpoint for Step 10000 at: /data/Raccoonbot_Openvla/openvla/openvla-runs/openvla-7b+raccoon_pick_place+b16+lr-0.0005+lora-
Max step 10000 reached! Stopping training...
11057

```

8. Inference Pipeline

Fine-tuned checkpoint를 사용하여 OpenVLA inference server를 실행하고, local MuJoCo client에서 simulation을 수행하였다.

8.1 OpenVLA Server 실행

```
cd /data/Raccoonbot_Openvla/openvla
```

```
CUDA_VISIBLE_DEVICES=0 python openvla_server.py ₩
```

```
--model_path /data/Raccoonbot_Openvla/openvla/openvla-runs/openvla-  
7b+raccoon_pick_place+b16+lr-0.0005+lora-r16+dropout-0.0--raccoon-grasp-push-  
300ep-10000step-b2-ga8-r16--image_aug ₩
```

```
--default-unnorm-key raccoon_pick_place ₩
```

```
--host 0.0.0.0 ₩
```

```
--port 8001 ₩
```

```
--device cuda
```

--default-unnorm-key raccoon_pick_place는 RaccoonBot dataset의 action normalization statistics를 사용하기 위해 지정하였다.

8.2 SSH Tunnel 연결

Local PC에서 remote OpenVLA server에 연결하기 위해 SSH tunnel을 사용하였다.

```
ssh -L 8001:127.0.0.1:8001 root@qlak315.iptime.org -p 11120
```

이후 local client는 다음 server URL로 접속하였다.

```
http://127.0.0.1:8001
```

8.3 Local MuJoCo Client 실행

Grasp task 실행 예시는 다음과 같다.

```
python openvla_grasp_push_4objects_custom_client_focus_pushboost.py ₩
```

```
--server_url http://127.0.0.1:8001 ₩
```

```
--xml_path Raccoon_colored_cylinder_cube.xml ₩
```

```
--task_type grasp ₩
```

```
--target_color red ₩
```

```
--target_shape cube ₩
```

```
--use_viewer ₩
```

```
--max_steps 40 ₩
```

```
--max_delta_xyz 0.01
```

Push task 실행 예시는 다음과 같다.

```
python openvla_grasp_push_4objects_custom_client_focus_pushboost.py ₩
```

```
--server_url http://127.0.0.1:8001 ₩
```

```
--xml_path Raccoon_colored_cylinder_cube.xml ₩
```

```
--task_type push ₩
```

```
--target_color blue ₩
```

```
--target_shape cube ₩
```

```
--use_viewer ₩
```

```
--max_steps 40 ₩
```

```
--max_delta_xyz 0.01
```

Push task에서는 cube가 더 명확하게 밀리도록 client-side에서 push target placement stabilization과 push boost를 적용하였다.

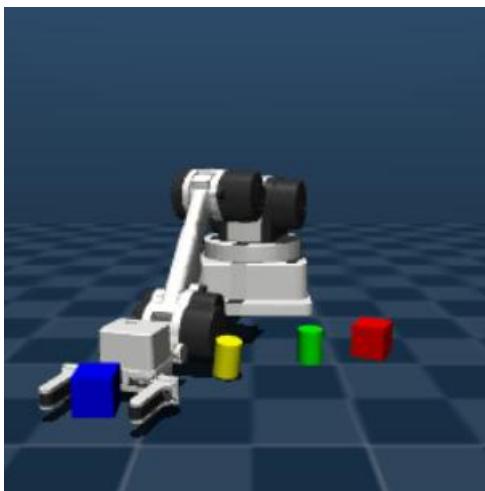
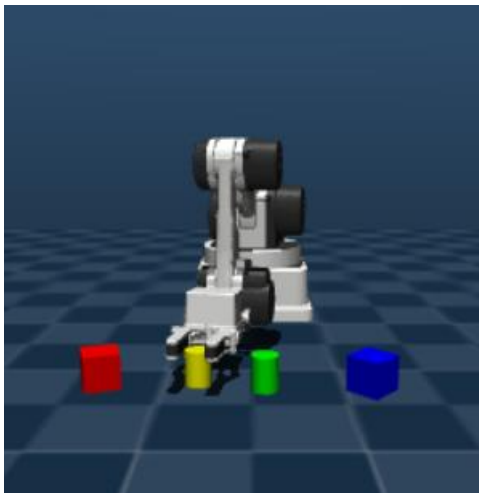
9. Episode Visualization

생성된 episode의 frame_*.png 이미지를 GIF로 변환하여 demonstration 동작을 시각화 하였다. 각 episode folder는 다음과 같은 구조를 가진다.

episode_000007/

|—— frame_000000.png
|—— frame_000001.png
|—— ...
|—— frame_000024.png
└—— meta.json

Grasp task에서는 로봇이 target object 쪽으로 이동한 뒤 gripper를 닫는 동작을 보였다.
Push task에서는 gripper를 open 상태로 유지하고 end-effector를 낮춘 뒤, positive y 방향으로 이동하여 cube를 미는 동작을 보였다.



10. Fine-tuned Model Local Inference Demo

본 프로젝트에서 직접 생성한 300개 demonstration dataset으로 OpenVLA LoRA fine-

tuning을 수행한 뒤, fine-tuned model을 local MuJoCo 환경에서 실행하였다.

이 결과는 dataset episode를 단순 재생한 것이 아니라, OpenVLA model이 현재 camera image와 language instruction을 입력받아 action을 예측하고, 그 예측 결과를 MuJoCo simulation에서 실행한 것이다.

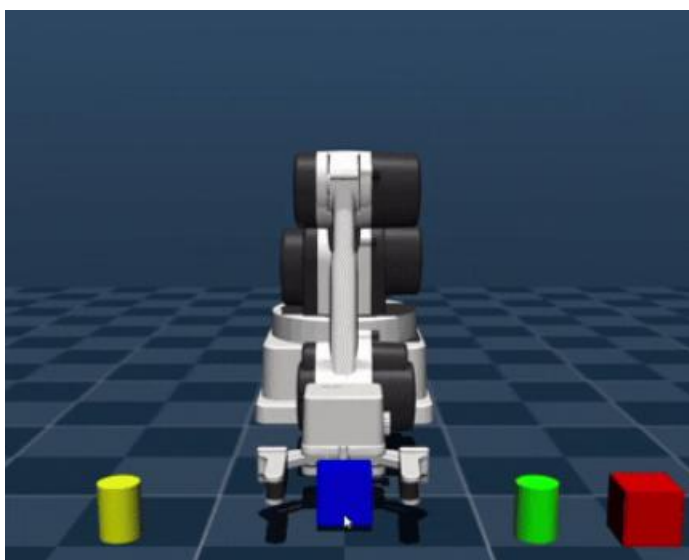
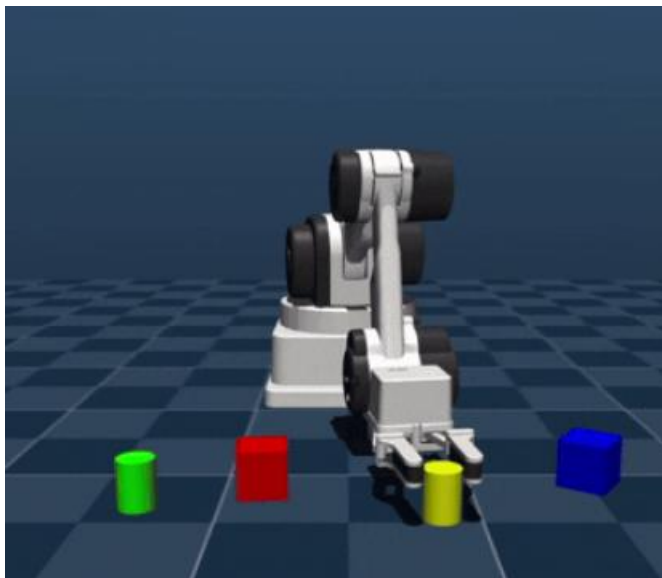
Local inference demo에서는 다음 task를 확인하였다.

Task Instruction

Result

Grasp grasp the yellow cylinder target cylinder 방향으로 이동 후 gripper close

Push push the blue cube gripper를 open 상태로 유지하고 cube를 앞으로 push



실행 결과, fine-tuned model은 language instruction에 따라 grasp와 push task를 구분하

여 수행하였다. Grasp instruction에서는 target object를 향해 이동하고 gripper를 닫는 동작이 나타났으며, push instruction에서는 gripper를 open 상태로 유지한 채 cube를 앞으로 미는 동작이 나타났다.

11. 실제 RaccoonBot Hardware Demo

추가로 fine-tuned OpenVLA model을 실제 RaccoonBot hardware와 연결하여 yellow cylinder grasp task를 수행하였다.

실제 hardware test는 다음과 같은 구조로 진행하였다.

1. Remote GPU server에서 fine-tuned OpenVLA inference server 실행
2. Local PC에서 SSH tunnel을 통해 server와 연결
3. Local PC에 실제 RaccoonBot hardware 연결
4. OpenVLA model이 camera image와 language instruction을 입력받아 action 예측
5. 예측된 action을 RaccoonBot joint motion으로 변환하여 실제 robot에 전송

실제 RaccoonBot 실행에는 `openvla_multicolor_client_real_robot.py`를 사용하였다. 이 client는 서버에서 받은 action을 먼저 MuJoCo environment 기준으로 처리한 뒤, target end-effector position을 실제 RaccoonBot의 joint angle로 변환하여 hardware에 전송한다.

실제 실행 task는 다음과 같다.

Item	Value
Task	Yellow cylinder grasp
Instruction	grasp the yellow cylinder
Robot	RaccoonBot
Connection	Local PC + RaccoonBot hardware
Server	Remote OpenVLA inference server
Result	yellow cylinder를 향해 이동 후 grasp 동작 수행

실제 RaccoonBot 실행 명령어는 다음과 같다.

```
python openvla_multicolor_client_real_robot.py ₩
```

```
--server_url http://127.0.0.1:8001 ₩
```

```
--target_color yellow ₩
```

```
--use_real_robot ₩
```

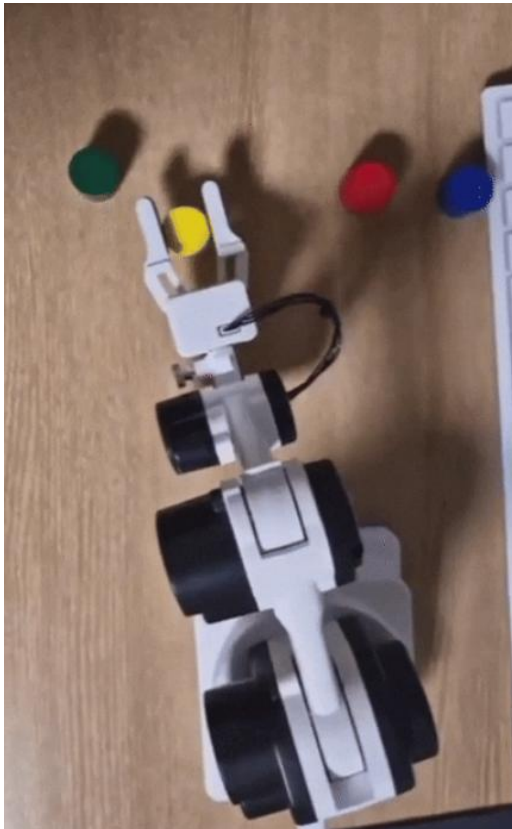
```
--use_viewer ₩
```

```
--max_steps 40 ₩
```

```
--max_delta_xyz 0.005 ₩
```

```
--speed 40
```

초기 테스트에서는 안전을 위해 max_steps, max_delta_xyz, speed 값을 작게 설정하여 짧은 이동부터 확인하였다. 이후 로봇이 정상적으로 동작하는 것을 확인한 뒤 step 수를 늘려 yellow cylinder grasp 동작을 끝까지 수행하였다.



실험 결과, fine-tuned OpenVLA model이 simulation 환경뿐만 아니라 실제 RaccoonBot hardware에서도 language instruction에 따라 target object를 향해 이동하고 yellow cylinder를 grasp하는 동작을 수행할 수 있음을 확인하였다.

12. Before / After 비교

본 프로젝트에서 기존 pipeline과 비교하여 변경된 점은 다음과 같다.

Item	Before	After
Object type	Colored cylinder only	Green/yellow cylinder + red/blue cube
Task type	Grasp only	Grasp + cube push
Language instruction	grasp the {color} cylinder	grasp the {color} {shape}, push the {color} {shape}
Active scene	Colored cylinders	4-object scene: 2 cylinders + 2 cubes
Dataset size	Original grasp dataset	300 successful demonstrations
Final task set	Cylinder grasp	4 grasp tasks + 2 cube push tasks
Local inference	Cylinder grasp 중심	Fine-tuned model로 grasp/push local inference 수행
Hardware demo	없음	실제 RaccoonBot으로 yellow cylinder grasp 수행

Code 측면에서는 기존 cylinder grasp demonstration generation에서 multi-object grasp 와 cube push demonstration generation이 가능하도록 확장하였다. 또한 local client에서는 task_type, target_color, target_shape를 인자로 받아 grasp와 push task를 선택적으로 실행할 수 있도록 구성하였다.

추가로 실제 RaccoonBot hardware demo를 통해, fine-tuned model의 action prediction 결과를 실제 robot motion으로 연결하는 과정까지 확인하였다.

13. 결과 및 분석

Fine-tuning 이후 inference 결과를 확인한 결과, 모델은 language instruction에 따라 grasp와 push task를 서로 다르게 수행하였다.

Grasp instruction에서는 target object 쪽으로 end-effector가 이동하고, gripper를 닫는 동작이 나타났다. 반면 push instruction에서는 gripper를 open 상태로 유지한 뒤, end-effector를 낮추고 target cube를 앞으로 미는 동작이 나타났다. 이는 dataset에 포함된

task type 차이가 fine-tuned OpenVLA model의 action prediction에 반영되었음을 의미한다.

다만 push task는 grasp task보다 안정성이 낮았다. 특히 cylinder push는 접촉 위치와 마찰, 굴림 특성에 따라 이동 방향과 거리가 불안정하였다. 이 때문에 최종 dataset에서는 cylinder push를 제외하고 cube push만 사용하였다.

또한 inference 과정에서 push 이동량이 작게 나타나는 경우가 있어, client-side에서 push target placement stabilization과 push action boost를 추가하였다. 이를 통해 push 동작이 더 명확하게 나타나도록 보정하였다.

실제 RaccoonBot hardware test에서는 yellow cylinder grasp를 수행하였다. 실제 로봇은 simulation과 달리 motor delay, joint error, 마찰, 물체 위치 오차 등의 영향을 받을 수 있기 때문에 처음에는 작은 step과 낮은 속도로 테스트하였다. 이후 정상 동작을 확인한 뒤, yellow cylinder를 향해 이동하고 grasp하는 동작을 끝까지 수행하였다.

이 결과를 통해 본 프로젝트는 단순히 MuJoCo simulation에서만 끝나는 것이 아니라, OpenVLA inference server와 local client, 그리고 실제 RaccoonBot hardware를 연결하는 전체 pipeline을 확인했다는 점에서 의미가 있다.

14. 한계점

본 프로젝트의 한계점은 다음과 같다.

첫째, cylinder push는 실험했지만 최종 dataset에서는 제외하였다. Cylinder는 cube와 달리 접촉 시 굴러가거나 옆으로 틀어지는 현상이 발생하여 안정적인 push demonstration을 확보하기 어려웠다.

둘째, push task는 grasp task보다 성공률과 동작 안정성이 낮았다. Push는 물체와의 접촉 위치, 접촉 높이, 마찰 조건에 민감하기 때문에 추가적인 trajectory tuning 또는 physics parameter 조정이 필요하다.

셋째, simulation과 실제 hardware 사이에는 차이가 존재한다. 실제 RaccoonBot에서는 motor delay, joint backlash, 물체 위치 오차, 바닥 마찰 등의 요인이 존재하기 때문에, simulation에서 잘 수행되는 action이 실제 환경에서 항상 동일하게 수행되지는 않을 수 있다.

넷째, dataset 규모가 300 episode로 제한되어 있어, 더 다양한 object placement와 language instruction에 대한 generalization은 제한적일 수 있다.

15. 결론

본 과제에서는 기존 RaccoonBot OpenVLA pipeline을 확장하여 cube object와 cube push task를 추가하였다. 이를 위해 MuJoCo scene을 수정하고, multi-object grasp 및 cube push demonstration을 생성하는 dataset generation code를 작성하였다. 최종적으로 4개의 active object와 6개의 task-target 조합으로 구성된 300개의 successful episode dataset을 생성하였다.

이후 raw MuJoCo episode를 RLDS intermediate format과 TFDS dataset으로 변환하고, openvla/openvla-7b base model을 LoRA 방식으로 10,000 step fine-tuning하였다. Fine-tuned model은 local MuJoCo client에서 language instruction에 따라 grasp와 push task를 구분하여 수행하는 동작을 보였다.

또한 실제 RaccoonBot hardware와 연결하여 grasp the yellow cylinder instruction에 대한 grasp 동작을 수행하였다. 이를 통해 본 프로젝트는 기존 cylinder grasp-only pipeline을 multi-object, multi-task setting으로 확장했을 뿐만 아니라, fine-tuned OpenVLA model의 inference 결과를 실제 RaccoonBot hardware 동작으로 연결하는 과정까지 확인하였다.

결과적으로 본 프로젝트는 RaccoonBot 환경에서 OpenVLA를 활용한 새로운 manipulation task adaptation 가능성을 확인하였으며, 향후 실제 robot에서 cube grasp와 cube push까지 확장하기 위한 기반을 마련하였다.